MineGuard

AI-enabled mine subsidence monitoring & early warning

Complete system reference

PROJECT	SUBSIDENCE-NET / MineGuard — Smart India Hackathon 2026
SCOPE	Wireless surface sensor mesh over an underground longwall panel, with server-side deformation reconstruction, risk scoring and multi-channel early warning
SENSING	LIS3DH IMU · vibration sensor · NEO-6M GNSS — *tilt is the only mechanical quantity measured; strain and subsidence are reconstructed from the array*
RADIO	Ebyte E220-900M22S (LLCC68, SPI) multi-hop LoRa mesh, 865–867 MHz
STACK	ESP32-S3 mini nodes · ESP32 mini gateway · FastAPI · TimescaleDB + PostGIS · React + Leaflet + Three.js
TESTS	217 passing — 32 protocol (incl. C ↔ Python byte identity), 122 simulation, 63 backend
GENERATED	9 September 2026, from the repository source

Contents

1 System overview

MineGuard detects ground subsidence over an active underground coal panel and warns before it damages surface structures. A battery-powered wireless mesh of sensor nodes sits on the surface above the panel; a gateway collects their frames and forwards them to a server that reconstructs the deformation field, scores risk, and pushes warnings to a dashboard, a mobile app and SMS.

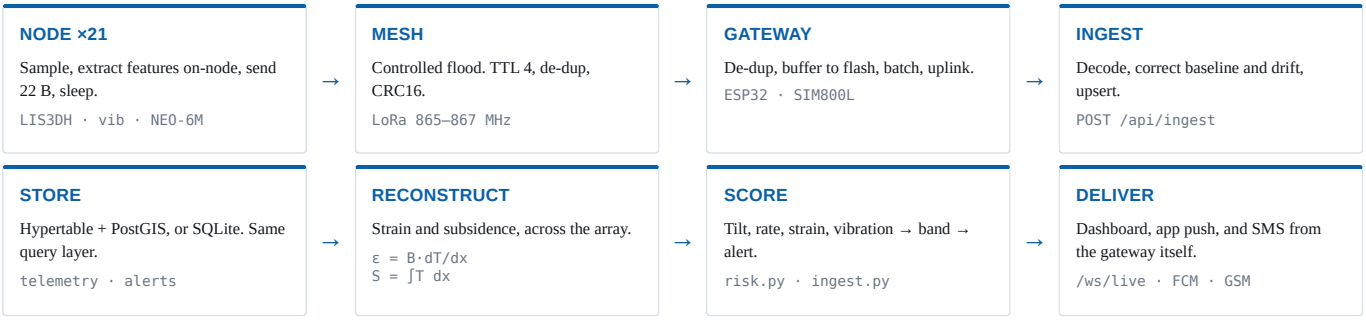
1.1 The central design decision

Each node measures exactly one mechanical quantity: **tilt**. It carries no crack gauge and no distance ranger. This is not a gap, because over a subsidence trough the quantities an engineer needs are the neighbouring derivatives of a single curve:

$$S = \int T\,dx \quad \cdot \quad T = dS/dx \quad \cdot \quad K = dT/dx \quad \cdot \quad U = B \cdot T \quad \cdot \quad \varepsilon = dU/dx = B \cdot K$$

A single node cannot produce strain. An *array* of nodes can: differentiate the tilt field and you get strain; integrate it and you get subsidence. That is what the server does, and it is validated against the analytic influence-function model rather than against its own output — strain correlation 0.995, subsidence within 2%, on a 21-node survey line.

1.2 Pipeline



Control flows the other way: a config downlink is queued by the dashboard, relayed by the gateway when the node next wakes, and stays `pending` until the node's ACK echoes the config hash.

1.3 What makes it degradable

Failure	What still works
One node dies	Mesh reroutes; neighbours still cover the ground; health alert raised
Gateway loses WAN	Sensing, mesh, flash buffering, **and local SMS alerting**
MQTT broker down	Everything — the gateway falls back to HTTP ingest
Backend down	Gateway buffers and reconciles later; gateway SMS still fires; the dashboard falls back to its built-in physics model and says so in the header
ML service down	Deterministic scoring, reconstruction and alerting, unchanged
Corrupt radio frames	Everything; frames counted as `rejected`, never fatal

2 Hardware

2.1 Sensor node (×21)

Component	Role	Notes
ESP32-S3 mini	MCU	Deep sleep between duty cycles; on-node FFT and averaging
LIS3DH accelerometer	Tilt, vibration, temperature	±2 g, 12-bit → ~1 mg/LSB. Static tilt from the gravity vector; vibration RMS and peak frequency from the high-rate FIFO; die temperature for drift correction
Vibration sensor	Wake interrupt	Wired to an EXT1 wake pin — a blast or impact wakes the node out of deep sleep at zero standing current, instead of waiting for its slot
NEO-6M GNSS	Position, UTC, tamper	~2.5 m CEP. Duty-cycled hard: it draws tens of mA whenever the antenna is live, against microamps asleep
E220-900M22S	LoRa radio	LLCC68 over **SPI** (the "M" suffix — register-level, not the "T" UART transparent variant). 22 dBm, 865–867 MHz. Gives per-packet RSSI/SNR and real channel-activity detection for cheap wake-on-radio
Solar panel + Li-ion	Power	`vbat_mv` in every frame; `TLM_LOW_BATTERY` flag below 3500 mV

> **Temperature is a measurement channel, not a convenience.** Thermal expansion of the mounting post shifts apparent tilt by ~18 mdeg/°C. Over a 15 °C day that is ≈4.7 mm/m — about 5.4× the sensor's own noise, and *systematic*, so averaging never removes it. Uncorrected, the system raises a false alarm every afternoon. The die temperature rides in every frame for exactly one purpose: subtracting that drift.

2.2 Gateway (×1)

Component	Role	Notes
ESP32 mini	MCU	De-duplicates, batches, buffers, relays downlinks
E220-900M22S	Mesh side	Same module as the nodes
SIM800L V2 (5 V)	SMS + GPRS	The alert channel that must never depend on anything else
Flash (LittleFS)	Store-and-forward	Frames are 34 B, so ~1 MB holds ~30 000 — days of buffer at a 60 s cadence

> **Gateway power is the likeliest field failure.** The SIM800L draws ~2 A bursts during GSM transmit. An ESP32 mini browning out mid-SMS will reset. Size the supply and bulk capacitance for the burst, not the average.

2.3 Field layout — two constraints, and the tighter one wins

Constraint	Limit	Consequence if violated
Radio connectivity	`≤ 241 m`	Frames do not reach the gateway
Strain resolution	`≤ 71 m`	Strain correlation collapses from 0.98 to 0.29 — tilt still correct, strain unusable
Integration anchor	`≥ 1·r beyond the panel edge`	Every subsidence depth in the row reads low; a node only 75 m out has itself already dropped ~341 mm

The sensing constraint binds about three times tighter than the radio. A field planned around radio range alone delivers every frame and reconstructs nothing. Both limits are computed in code — `max_sensing_spacing_m()` against `RadioModel.max_reliable_spacing_m()` — and the negative result is asserted in the test suite so the node count cannot quietly be optimised down.

> **21 nodes as a line, not a grid.** Covering the whole panel as a grid at 71 m spacing needs ~105 nodes. The same 21 arranged as a single **survey transect** running a full radius of influence past each panel edge resolve the complete profile: strain correlation 0.995, subsidence within 2%, both ends properly anchored. This is also how subsidence has been monitored for a century — survey lines, not grids.

3 Repository map

Line counts from source. Vendored dependencies, build output and caches omitted.

3.1 `packages/subnet-proto` — the shared wire protocol

File	Lines	Responsibility
`subnet_proto/proto.py`	509	The codec. Header, seven payload types, CRC16, encode/decode. Imported by both backend and simulator so they cannot drift
`subnet_proto/__init__.py`	24	Public surface
`tests/test_proto.py`	269	Round trips, corruption handling, engineering-unit conversions, C ↔ Python byte identity
`tests/proto_check.c`	70	Compiles the firmware header and prints its view of the wire format for Python to diff

3.2 `firmware` — the C side of the same contract

File	Lines	Responsibility
`common/mesh_proto.h`	183	Packed structs, message and event codes, flag bits, CRC — byte-for-byte mirror of `proto.py`
`node/src, gateway/src`	–	NOT BUILT The protocol and frame layout are already fixed, so the contract these must satisfy is settled

3.3 `backend` — FastAPI service

File	Lines	Responsibility
`app/main.py`	96	App, lifespan, CORS, the `/ws/live` WebSocket
`app/api.py`	386	All 17 REST endpoints
`app/ingest.py`	499	Frame decode and dispatch, baselines, tilt rates, the per-batch field pass, alert raising, GNSS displacement
`app/deformation.py`	238	NEW Strain and subsidence reconstructed from the tilt array; the spacing rule
`app/risk.py`	165	Baseline + thermal correction, scoring, NCB damage bands
`app/state.py`	352	Builds the snapshot document served by REST and pushed over the socket
`app/models.py`	186	SQLAlchemy table definitions (8 tables)
`app/mqtt.py`	119	Optional broker bridge: uplink subscribe, downlink publish
`app/broadcaster.py`	62	Coalesces snapshot rebuilds to ≤4 Hz
`app/hub.py`	57	WebSocket subscriber fan-out
`app/db.py`	97	Async engine, session factory, schema bootstrap
`app/config.py`	61	Environment-driven settings and thresholds
`migrations/001_schema.sql`	239	TimescaleDB hypertables + PostGIS geometry for the deployed stack
`tests/test_api.py`	358	Ingest, baselines, thermal drift, reconstruction, GNSS, alerts, downlink
`tests/test_deformation.py`	270	NEW Reconstruction validated against the analytic physics

3.4 `ml` — physics, sensor model and the virtual gateway

File	Lines	Responsibility
`simulator/physics.py`	261	Influence-function (Knothe) subsidence model with analytic derivatives
`simulator/sensors.py`	273	What the hardware actually reports: install offsets, thermal drift, quantisation, GNSS noise
`simulator/scenarios.py`	369	Five labelled scenarios — stable, slow creep, accelerating, sudden collapse, false alarm
`simulator/mesh.py`	289	LoRa link budget, obstructions, flood routing, delivery statistics
`simulator/field.py`	247	Node layout: grid and transect builders, the sensing-spacing rule
`simulator/virtual_gateway.py`	352	Encodes genuine frames, routes them through the mesh, posts them exactly as an ESP32 would
`service/, training/`	–	NOT BUILT Port and `ML_SERVICE_URL` already wired in compose

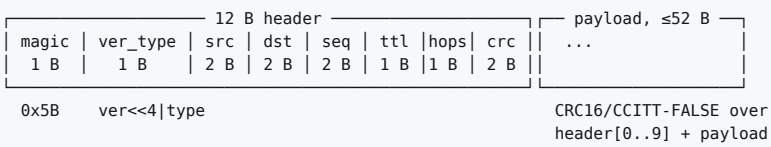
3.5 `web` — React dashboard

File	Lines	Responsibility
`src/data/createSource.ts`	34	Probes the backend; falls back to the built-in simulator if it does not answer
`src/data/liveSource.ts`	148	WebSocket transport with backoff; REST for history. Does no interpretation
`src/data/types.ts`	110	The snapshot contract, mirroring `state.py`
`src/sim/feed.ts`	439	Offline physics simulator — the fallback when the backend is unreachable
`src/components/map/`	1063	Leaflet 2-D map, Three.js 3-D subsidence terrain, heat layer, tiles
`src/components/charts/`	436	Deformation trend (tilt / vibration / temperature), prediction chart
`src/components/ (rest)`	~830	KPI row, alerts panel, gauges, layout, icons

4 Wire protocol

One codec, written in Python (`packages/subnet-proto`) and mirrored in C (`firmware/common/mesh_proto.h`). The conformance suite compiles the real firmware header and asserts both encoders emit byte-identical frames, so firmware and backend cannot silently drift apart. All frames are little-endian and packed — native ESP32 layout, no byte swapping.

4.1 Frame structure



Field	Type	Meaning
`magic`	u8	`0x5B` . First cheap rejection of noise
`ver_type`	u8	High nibble protocol version (`0x1`), low nibble message type
`src`	u16	Originating node address. `0x0001` is the gateway, `0xFFFF` broadcast, `0x0000` unassigned
`dst`	u16	Destination; telemetry is addressed to the gateway
`seq`	u16	Per-source counter. With `src` it drives flood de-duplication
`ttl`	u8	Decremented each hop, dropped at 0. Default 4
`hops`	u8	Incremented each hop — the server learns the route actually taken
`crc16`	u16	CCITT-FALSE, poly 0x1021, init 0xFFFF. Never trust the radio

4.2 Message types

Type	Name	Dir	Payload	Frame	Purpose
`0x1`	TELEMETRY	up	`22 B`	`34 B`	Periodic sensor frame, on the duty cycle
`0x2`	EVENT	up	`14 B`	`26 B`	Node-side breach — **bypasses the duty cycle**
`0x3`	CONFIG_SET	down	`20 B`	`32 B`	Remote reconfiguration
`0x4`	CONFIG_ACK	up	`9 B`	`21 B`	Echoes the config hash; closes the loop
`0x5`	NEIGHBOR	up	`5 B + 4/entry`	`17 B +`	RSSI table → topology graph. Up to 8 neighbours
`0x6`	TIME_SYNC	down	`6 B`	`18 B`	Clock discipline; nodes have no RTC across deep sleep
`0x7`	POSITION	up	`17 B`	`29 B`	GNSS fix, low rate

4.3 TELEMETRY — 22 B

```
struct __attribute__((packed)) {
    uint32_t t_epoch;       /* unix seconds, disciplined by MSG_TIME_SYNC     */
    int16_t  pitch_mdeg;    /* milli-degrees, gravity vector from LIS3DH      */
    int16_t  roll_mdeg;     /* milli-degrees                                  */
    uint16_t vib_rms_mg;    /* RMS acceleration over the sample window, mg    */
    uint16_t vib_peak_hz;   /* dominant frequency, on-node FFT                */
    int16_t  temp_c_x100;   /* LIS3DH die temp, centi-C — drift correction    */
    uint8_t  n_samples;     /* raw samples averaged into this frame           */
    uint8_t  gnss_status;   /* [1:0] fix quality, [7:2] satellite count       */
    uint16_t vbat_mv;       /* battery millivolts                             */
    int8_t   rssi;          /* last received RSSI, dBm                        */
    uint8_t  snr;           /* last received SNR, (dB + 20) * 4               */
    uint8_t  flags;         /* see TLM_FLAG_*                                 */
    uint8_t  reserved;
} tlm_t;                                        /* struct format: <IhhHHhBBHbBBB */
```

`n_samples` exists so the server knows this reading's noise (σ/√n) *before* it differentiates the tilt field — differentiation amplifies noise, so the estimator needs to know how much it is working with.

4.4 Other payloads

```
EVENT      <IBBii    t_epoch, event_code, severity, value, threshold
CONFIG_SET <HHHBHHHhhBH
                     cfg_version, sample_interval_s, wor_period_ms, tx_power_dbm,
                     tilt_alert_mdeg, vib_alert_mg, tilt_rate_alert_mdeg_h,
                     tilt_offset_pitch, tilt_offset_roll, flags, cfg_hash
CONFIG_ACK <IHHB     t_epoch, cfg_version, cfg_hash, status
NEIGHBOR   <IB + n×  t_epoch, count, then (addr:u16, rssi:i8, snr:u8) per neighbour
TIME_SYNC  <IH       t_epoch, t_millis
POSITION   <IiihHB   t_epoch, lat_e7, lon_e7, alt_m, h_acc_cm, gnss_status
```

4.5 Enumerations and flags

Set	Values
Event codes	`0x01 TILT_RATE · 0x02 TILT_ABSOLUTE · 0x03 TILT_ACCEL · 0x04 VIBRATION · 0x05 DISPLACEMENT · 0x06 NODE_TAMPER · 0x07 LOW_BATTERY`
Severity	`0 INFO · 1 WARNING · 2 HIGH · 3 CRITICAL`
Telemetry flags	`0x01 TILT_FAULT · 0x02 GNSS_FAULT · 0x04 VIB_FAULT · 0x08 UNCALIBRATED · 0x10 LOW_BATTERY · 0x20 RELAYED`
Config flags	`0x01 RELAY_ENABLED · 0x02 GNSS_ENABLED · 0x04 VIB_ENABLED · 0x08 DEEP_SLEEP · 0x10 RECALIBRATE`
GNSS fix quality	`0 NO_FIX · 1 FIX_2D · 2 FIX_3D · 3 FIX_DGPS` (low 2 bits of `gnss_status`; satellite count in the high 6)
Config status	`0 APPLIED · 1 REJECTED · 2 PARTIAL`

Why `TILT_ACCEL` and `DISPLACEMENT` mean what they do. With no crack gauge, `0x03` was repurposed from `CRACK_OPEN` to tilt *acceleration* — the strongest single-node precursor. `0x05 DISPLACEMENT` keeps its name but is now sourced from GNSS rather than a ranger: a node that has moved metres is a collapse or a theft.

5 How data is received

Four distinct paths. Three carry data inward; one carries control back out.

5.1 Path A — HTTP ingest (default)

The simplest path for a constrained ESP32 that already has a TLS-free WiFi stack, and the one the virtual gateway uses. The gateway batches up to 256 frames (64 by default), base64-encodes each, and POSTs them.

```
POST /api/ingest
Content-Type: application/json

{
  "frames": ["W xEAA...", "W xEAB...", ...],   // base64 of raw protocol bytes
  "gateway": "gw-01",                          // optional, informational
  "site": "jharia"                             // optional; first site if omitted
}

200 OK
{ "accepted": 62, "rejected": 2, "alertsRaised": 1,
  "errors": ["bad CRC: got 0x1A2B, computed 0x7F09", "..."] }   // at most 5
```

Note what the response does *not* do: a corrupt frame does not fail the batch. Corrupt radio frames are expected, not exceptional — they are counted, sampled into `errors`, and the good frames alongside them still land.

5.2 Path B — MQTT (deployed shape)

Direction	Topic	Payload
Gateway → server	`subnet/gw/+/up`	Raw frame bytes, or newline-separated base64 for a batch
Server → gateway	`subnet/gw/{id}/cmd`	A single `CONFIG_SET` frame

The bridge (`app/mqtt.py`) subscribes on startup and reconnects with a 5 s retry on loss — a field deployment loses its broker regularly, so this is logged at warning and never fatal. If `MQTT_HOST` is unset the bridge simply does not start and gateways use Path A. **Losing the broker degrades the transport, never the system.**

Both paths converge on the same function, `ingest_frames()`. There is exactly one decode path, so a bug cannot exist on one transport and not the other.

5.3 Path C — WebSocket out to clients

```
ws://host/ws/live

  server → client   { ...full snapshot... }     immediately on connect
  server → client   { ...full snapshot... }     on every change, coalesced to ≤4 Hz
  server → client   { "type": "ping" }          every 25 s idle, so proxies do not drop it
```

The first message after any connect or reconnect is a **full snapshot, not a delta**. A client that dropped for an hour is immediately correct rather than applying updates to stale state — which matters on a site where connectivity comes and goes.

5.4 Path D — control downlink

```
Dashboard ──POST /api/nodes/{addr}/config──►  node_configs row, status='pending'
                                                        │
                                       CONFIG_SET frame ──┤ MQTT subnet/gw/+/cmd
                                                        ▼
                                              Gateway holds it until the node
                                              wakes (wake-on-radio, 2 s period)
                                                        ▼
                                              Node applies, computes cfg_hash
                                                        │
          status='applied' ◄──── CONFIG_ACK (echoes cfg_hash) ────┘
```

A config stays `pending` until the node's ACK echoes the hash. That round trip is the point: a command that never reached a sleeping node must be *visible* as such, not assumed. States: `pending → sent → applied | rejected | timeout`.

5.5 What happens to each frame type on arrival

Frame	Action
`TELEMETRY`	Baseline- and temperature-correct → compute tilt → upsert into `telemetry` → update `nodes.last_seen`. First frame from an unbaselined node *establishes* its baseline (attitude and temperature) and raises nothing
`EVENT`	Insert into `events` → raise an alert immediately; the node already decided this was a breach
`POSITION`	If the node was never surveyed, self-place it. Otherwise compare against the recorded position and raise `Node Displaced` beyond 15 m. Unusable fixes (2-D only, or no accuracy estimate) are ignored
`CONFIG_ACK`	Mark the `node_configs` row applied/rejected/partial; update `nodes.active_cfg_version`
`NEIGHBOR`	Insert `mesh_links` rows that feed the live topology graph
`CONFIG_SET / TIME_SYNC`	Downlink types. A node echoing one back is harmless and ignored

5.6 Idempotency

The protocol timestamps in whole seconds, so two frames from one node can legitimately share a primary key. Telemetry is **upserted**, not ignored: a relayed duplicate rewrites identical values and is therefore harmless, while a genuinely newer reading in the same second replaces the older one. Dropping the second frame instead would discard real measurements silently, which is the worst way for a monitoring system to fail.

6 Ingest pipeline

```
ingest_frames(session, settings, frames, site_slug)
  │
  ├─ resolve site  ──────────────────── none? reject the whole batch, say why
  │
  ├─ FOR EACH FRAME
  │    ├─ decode()  ── ProtocolError? count as rejected, log, CONTINUE
  │    │                (bad magic · bad version · bad CRC · wrong length)
  │    └─ _handle()  ── dispatch on payload type (§5.5)
  │         └─ any exception? count as rejected, log, CONTINUE
  │
  ├─ IF anything was accepted:  _field_pass()          ← once per batch
  │    ├─ load every node + its latest telemetry
  │    ├─ correct each for baseline and temperature
  │    ├─ estimate_field()   → strain and subsidence across the array
  │    ├─ _tilt_rates()      → °/h per node over a 6 h window
  │    ├─ assess() per node  → score, band, damage class
  │    └─ _raise_alert() for high/critical, subject to a 20 min per-node cooldown
  │         └─ exception here? logged; telemetry is still retained
  │
  └─ commit, return {accepted, rejected, alertsRaised, errors}
```

6.1 Why alerting is per batch, not per frame

Strain is a property of the field, not of a node — it comes from how tilt varies *between* nodes. So it cannot be judged frame by frame. Assessing per frame would also raise twenty near-identical alerts, one per node, for what is plainly a single advancing face.

6.2 Baselines

A node is hand-planted on uneven ground, so its raw attitude mostly describes how the post was hammered in. Its first frame becomes its commissioning baseline — **attitude and the temperature that attitude was measured at** — and every later reading is measured against it. Without this the system alerts on every node the moment it joins, then goes blind to the movement it exists to detect. `POST /api/nodes/{addr}/recalibrate` clears the baseline so the next frame establishes a new one; it is for use after a node is *legitimately* disturbed — re-planted, knocked by plant, re-levelled.

6.3 Auto-provisioning

A node that appears on the mesh without being provisioned is created *unplaced*. A node reporting from an unknown position is still better than a node silently discarded — but it takes no part in the field reconstruction, because the reconstruction divides by the distance between nodes and an unplaced node has no baseline to differentiate along.

7 Field reconstruction

`backend/app/deformation.py`. This stage exists because a node measures tilt and nothing else mechanical.

7.1 The identities

```
subsidence   S           = ∫ T dx        ← integrate the array
tilt         T  = dS/dx                  ← measured directly by the LIS3DH
curvature    K  = dT/dx   = d²S/dx²      ← differentiate the array
horiz disp   U  = B · T                  ← scale;  B ≈ 0.4 · H
strain       ε  = dU/dx   = B · K        ← differentiate the array
```

The ranger used to measure `U`, which is 60× the tilt at 150 m depth — strictly redundant with the IMU. The crack gauge used to measure `ε` at one point, which the tilt gradient across node pairs gives field-wide. Both were measuring quantities the array already contains.

7.2 Method

Step	How
Group	Nodes cluster into rows (constant y) and columns (constant x) with a 40 m tolerance — a crew plants posts by hand and by eye
Differentiate	**Central differences** along each row and column; one-sided at the ends. Strain = B × gradient
Integrate	Trapezoidal along each row, anchored at the row's outermost node
Govern	The reported strain is whichever axis has the larger magnitude, *with its sign* — tension opens cracks, compression buckles and shears, and the damage is not interchangeable

> **Why central differences and not a least-squares fit over a wide neighbourhood.** A wide fit looks more robust and is much worse here. The trough reverses curvature within about half a radius of influence, so a fit spanning that averages tension and compression together and reports neither. Measured: a 400 m fit window gave strain correlation 0.18; local central differences at adequate spacing give 0.98.

7.3 The two layout rules, measured

Strain recovery against the analytic model, whole-panel grid, full draw margin:

Spacing	Nodes	Strain corr.	Subs. corr.	Verdict
188 m	24	0.295	0.992	Strain unusable
150 m	35	0.695	0.997	Marginal
94 m	88	0.898	1.000	Acceptable
71 m	165	**0.982**	1.000	**The rule: r ÷ 3**
50 m	315	0.993	1.000	Diminishing returns

And anchor placement, which governs subsidence rather than strain. At 71 m spacing:

Anchor position	True subsidence there	Depth error	Flag
75 m beyond the panel edge (0.35·r)	341 mm	37%	`subsidenceValid: false`
214 m beyond the edge (1·r)	11 mm	2%	`subsidenceValid: true`

7.4 The transect result

A single survey line, running one full radius of influence past each panel edge:

Nodes	Spacing	Strain corr.	Subs. error	Anchored
11	103 m	0.941	5%	yes
15	73 m	0.981	3%	yes
21	**51 m**	**0.995**	**2%**	**yes**
25	43 m	0.998	2%	yes

7.5 Degraded honesty

Two validity flags travel with every node, and both exist because **zero reads as safe**:

- `strainValid: false` — the node had no neighbour to take a gradient against (unplaced, isolated, or coincident with another). It reports tilt honestly and strain not at all, rather than reporting a strain of zero.
- `subsidenceValid: false` — this node's row has no anchor outside the trough, so the integration constant is unknown. The *shape* of the profile is still right; its absolute depth is not, and will read low.

8 Risk scoring and alerting

8.1 Corrections applied before anything is judged

```
tilt = (raw − baseline) − (temp − baseline_temp) × drift_coefficient

  baseline     removes how the post was hammered in
  temperature  removes the post's thermal expansion, ~18 mdeg/°C
               (when no temperature is available the correction is SKIPPED,
                not guessed — a wrong correction is worse than none)
```

8.2 The score

$$\text{score} = 0.90 \times \max\left(\frac{\text{tilt}}{\text{lim}}, \frac{\text{rate}}{\text{lim}}, \frac{\text{strain}}{\text{lim}}\right) + 0.10 \times \min\left(\frac{\text{vib}}{\text{lim}}, 1\right)$$

The governing term is whichever criterion sits closest to its limit. Summing them would let two comfortable readings average away one dangerous one.

Criterion	Default limit	Source
Tilt	0.60°	Directly measured. 10 mm/m, the NCB-style disruptive limit
Tilt rate	0.05 °/h	Fitted over a 6 h window. **The precursor** — crosses its limit while absolute tilt is still comfortably inside its own
Strain	3.00 mm/m	Reconstructed from the array. NCB “appreciable damage” boundary
Vibration	400 mg	Corroborating only, at a tenth of the weight

8.3 Bands and damage classes

Risk band	Score	Damage class	Strain (mm/m)
low	< 0.35	negligible	< 0.5
medium	< 0.60	slight	< 1.5
high	< 0.85	appreciable	< 3.0
critical	≥ 0.85	severe	< 6.0
		very_severe	≥ 6.0

Damage bands are keyed on strain *magnitude* — compression damages structures too. Tilt above 10 mm/m independently disrupts services, drainage and rail, and lifts the classification one step without ever lowering it. When strain could not be reconstructed the class is based on tilt alone and is a floor, not a verdict.

8.4 Alert sources and naming

Source	Path	Latency
Node-side threshold breach	EVENT → _event	Fastest — bypasses the duty cycle
Server-side high/critical	_field_pass → _raise_alert	Once per batch
Node physically displaced	POSITION → _position	On the next GNSS report
Node health	last_seen > 180 s	At snapshot build

Each alert is named after the criterion that actually drove it — **Tilt Rate Exceeded**, **Ground Strain Exceeded**, **High Deformation Detected**, **Abnormal Tilt Detected**, **Node Displaced** — and carries `hours_to_threshold`, a straight-line extrapolation of the current tilt rate to the disruptive limit. Subsidence accelerates, so that figure is optimistic and should read as “no sooner than”. It is still the number an operator plans around, which is why it is stated rather than hidden inside a score.

> **Cooldown: 20 minutes per node.** An early-warning system that repeats itself every few seconds gets muted by the people it is meant to warn, and a muted system is worse than no system. Alert state is `open → acked → resolved`, with `acked_by` and timestamps recorded — acknowledgement is a human act against a name, because that is what an incident review will ask for.

8.5 Delivery

Channel	Transport	Survives
SMS	Gateway's own SIM800L	**No internet required.** The gateway's local rule engine can fire on a critical EVENT with no server involved. `alerts.notified_sms` is the idempotency flag between the local and server-directed paths
Android app	FCM push; REST + WS, or direct to the gateway over the site LAN	No WAN, via `gateways.lan_ip`
Dashboard	WebSocket	Falls back to its built-in physics model and says so in the header
Email	SMTP	Record, not warning

9 Database schema

Eight tables. SQLite locally, TimescaleDB + PostGIS deployed — the query layer is identical either way, and only the hypertable DDL differs. “Cloud vs local” is a deployment choice, not an architecture choice, which matters for a mine site with no reliable backhaul.

9.1 `sites` — one monitored panel

Column	Type	Meaning
`id`	int PK	
`slug, name, coalfield`	text	Identity
`origin_lat, origin_lon`	float	Local ENU metres are referenced to this
`seam_depth_m`	float	**H** — drives the influence radius and B
`extraction_thickness_m`	float	**m**
`subsidence_factor`	float	**a**, where S_max = a·m
`angle_of_draw_deg`	float	**β** — r = H / tan β
`face_x_m`	float	Where the working face has reached
`face_advance_m_per_day`	float	
`panel_x_start/x_end/y_min/y_max`	float	Extraction outline (PostGIS polygon in the deployed schema)

9.2 `nodes`

Column	Type	Meaning
`id, site_id, addr, label, zone`	—	Identity; `addr` is the 16-bit mesh address, unique per site
`lat, lon, x_m, y_m`	float	Position. A node without x/y takes no part in the reconstruction
`hw_revision`	text	`esp32s3-lis3dh-neo6m-e220-v2`
`baseline_pitch_mdeg`	int	Commissioning attitude
`baseline_roll_mdeg`	int	
`baseline_temp_c_x100`	int	**The reference the thermal correction is relative to.** A baseline without one is half a baseline
`position_source`	text	`survey` or `gnss`. A self-fix is good to metres — enough to place a pin, not a survey record
`position_acc_m`	float	Reported horizontal accuracy
`last_seen, is_active, active_cfg_version`	—	Health and config state

9.3 `telemetry` — hypertable, PK `(node_id, time)`

Column	Type	Meaning
`time, node_id`	—	Composite key; upserted on conflict
`pitch_mdeg, roll_mdeg`	int	Raw as reported
`tilt_mdeg`	float	Derived magnitude, denormalised for fast queries and for the rate fit
`vib_rms_mg, vib_peak_hz`	int	
`temp_c_x100`	int	Die temperature
`n_samples`	int	Averaging depth behind this reading
`gnss_status`	int	Packed fix quality + satellite count
`vbat_mv, rssi, snr_db, flags`	—	
`hops, seq`	int	From the header — the route actually taken

9.4 `alerts`

Column	Type	Meaning
`site_id, node_id, raised_at`	—	
`severity`	int	0 info … 3 critical
`category`	text	`threshold` | `anomaly` | `forecast` | `health`
`title, detail`	text	Named after the driving criterion; detail carries the zone
`tilt_deg, strain_mm_per_m, tilt_rate_deg_per_h, vibration_mg`	float	The readings at the moment it fired
`damage_class`	text	Null when strain could not be reconstructed
`hours_to_threshold`	float	**Lead time**
`state, acked_by, acked_at, resolved_at`	—	`open → acked → resolved`
`notified_sms`	bool	Idempotency between the gateway-local and server-directed SMS paths

9.5 Remaining tables

Table	Contents
`events`	Raw node-side breaches: `time, node_id, event_code, severity, value, threshold, received_at`
`node_configs`	One row per pushed config version: the nine settable fields, `cfg_hash`, `status`, `created_by`, and the sent/applied timestamps. Unique on `(node_id, cfg_version)`
`mesh_links`	`time, site_id, src_addr, dst_addr, rssi, snr_db` — from NEIGHBOR frames; feeds the topology graph
`gateways`	`slug, label, lan_ip, firmware, last_seen, battery_mv`. `lan_ip` is the Android app's direct-over-router path

10 REST API reference

Base path `/api`. Interactive docs at `/docs`. All responses JSON. 17 endpoints.

10.1 Health and live view

Endpoint	Detail
`GET /api/health`	→ `{status, dialect, time}`. `dialect` tells you whether you are on SQLite or Postgres. The dashboard probes this to decide live vs offline mode
`GET /api/snapshot ?site=<slug>`	The entire live view in one document — the same shape the WebSocket pushes (§11). Returns an empty snapshot rather than an error when no site exists yet
`GET /api/stats`	→ `{framesLast24h, framesTotal}`
`GET /`	→ `{service, docs, live}`

10.2 Sites

Endpoint	Detail
`GET /api/sites`	All site rows
`PATCH /api/sites/{slug}/face`	Body `{face_x_m: ≥0}`. Operator input: advancing the face re-drives the deformation model. Marks the snapshot dirty. 404 if the slug is unknown

10.3 Nodes

Endpoint	Detail
`GET /api/nodes`	All node rows, ordered by address
`PATCH /api/nodes/{addr}`	Body `{label?, zone?, lat?, lon?, x_m?, y_m?}` — where an operator dropped the pin. 400 if no fields, 404 if unknown
`POST /api/nodes/{addr}/recalibrate`	Clears the commissioning baseline so the next frame establishes a new one. For a node legitimately disturbed — re-planted, knocked by plant, re-levelled
`GET /api/nodes/{addr}/history ?range=1H|6H|24H|7D`	→ `[{t, pitch, roll, vib, tempC}]`, oldest first. **Baseline- and temperature-corrected**, so the chart and the gauges cannot disagree about the same node. Sample counts: 4 / 24 / 96 / 672

10.4 Alerts

Endpoint	Detail
`GET /api/alerts ?state=&limit=<≤500>`	Newest first, default 50
`POST /api/alerts/{id}/ack`	Body `{by: "operator"}`. Records who acknowledged and when
`POST /api/alerts/{id}/resolve`	Resolved alerts leave the live view

10.5 Configuration downlink

Endpoint	Detail
`GET /api/nodes/{addr}/config`	Last 10 config versions with their status
`POST /api/nodes/{addr}/config → 201`	Queues a downlink; returns `{addr, cfg_version, cfg_hash, status, transport}`. The hash is computed with the same codec the node will verify against

```
ConfigPush          range              default   meaning
  sample_interval_s   5 … 3600           60      duty cycle, seconds
  wor_period_ms       250 … 10000       2000     wake-on-radio listen cadence
  tx_power_dbm        10 … 22             22
  tilt_alert_mdeg     10 … 32000        2000     node-side absolute tilt trigger
  vib_alert_mg        10 … 60000         500
  tilt_rate_alert_mdeg_h  1 … 60000      150     node-side PRECURSOR trigger
  tilt_offset_pitch   ±32767              0      zero-offset calibration
  tilt_offset_roll    ±32767              0
  flags               0 … 255            15      relay|gnss|vib|deep_sleep|recalibrate
  created_by          —           "operator"
```

10.6 Commissioning and ingest

Endpoint	Detail
`POST /api/provision → 201`	**Idempotent.** Re-running a commissioning script is safe, and a gateway can re-announce its field after a rebuild without duplicating anything. Returns `{site, siteId, nodesCreated, nodesUpdated}`
`POST /api/ingest`	The uplink. Body `{frames: [base64], gateway?, site?}`, 1–256 frames. 400 only if the base64 itself is malformed — bad *frames* are counted, not fatal

```
ProvisionRequest                default    NodeSpecIn (per node)
  slug, name              required           addr           required, 1…0xFFFE
  coalfield                                  label          required
  origin_lat, origin_lon  required           zone
  seam_depth_m               150             lat, lon
  extraction_thickness_m       3.0           x_m, y_m
  subsidence_factor            0.65          baseline_pitch_mdeg
  angle_of_draw_deg           35             baseline_roll_mdeg
  face_x_m                     0             baseline_temp_c_x100
  face_advance_m_per_day      12
  panel_x_start / x_end        0 / 600
  panel_y_min / y_max       −200 / 200
  nodes                        []
```

Supplying the three baselines explicitly is how a real commissioning survey enters the system — the node's attitude on undisturbed ground *before* the face arrived, and the temperature it was measured at. Without them a node baselines itself against whatever deformation already exists the first time it reports, silently zeroing out the signal.

11 WebSocket and the snapshot document

One shape, served two ways: as JSON from `GET /api/snapshot` and pushed over `/ws/live`. It mirrors the `Snapshot` type in the web app exactly, so the dashboard swaps from its built-in simulator to the live backend by changing which data source it constructs and nothing else.

```
{
  "t": 1789085117000,          // server time, ms
  "day": 41.25,                // simulated day = face_x / advance rate
  "faceX": 495.0,              // where the working face has reached, m

  "nodes": [{
    "addr": 16, "id": "01", "label": "T-01", "zone": "Panel A - West",
    "lat": 23.7481, "lon": 86.4180, "x": -214.0, "y": 0.0,
    "isEdge": false, "online": true,

    "tiltPitchDeg": -0.043,    // baseline- AND temperature-corrected
    "tiltRollDeg":   0.021,
    "tiltDeg":       0.048,
    "tiltRateDegPerH": 0.0021, // fitted over a 6 h window — the precursor
    "vibrationMg":  14,
    "tempC":        29.91,     // post temperature, for the drift correction

    "subsidenceMm":  0.0,      // integrated along the transect
    "subsidenceValid": true,   // false = no anchor outside the trough
    "strainMmPerM":  1.84,     // reconstructed from the tilt gradient
    "strainValid":   true,     // false = array too sparse; NOT zero strain

    "riskScore": 0.31, "risk": "low", "damage": "slight",
    "gnssSats": 10, "hops": 2, "rssi": -86, "batteryPct": 78
  }, ...],

  "links":  [{ "a": 16, "b": 17, "rssi": -84.2, "onRoute": true }, ...],
  "alerts": [{ "id": "42", "severity": "high", "title": "Ground Strain Exceeded",
               "nodeId": "06", "zone": "Panel A - Center", "ts": 1789084000000,
               "metrics": [{ "label": "Tilt",      "value": "0.52°" },
                           { "label": "Strain",    "value": "+3.21 mm/m" },
                           { "label": "Vibration", "value": "Normal" }] }, ...],
  "prediction": [{ "t": 0, "actual": 0.12, "predicted": 0.12 }, ...,
                 { "t": 8, "actual": null, "predicted": 0.71 }, ...],

  "kpis": {
    "totalNodes": 21, "activeNodes": 21, "inactiveNodes": 0,
    "totalAlerts": 8, "criticalAlerts": 3, "highAlerts": 5,
    "maxTiltDeg": 0.518,          "tiltThresholdDeg": 0.60,
    "maxTiltRateDegPerH": 0.0025, "tiltRateThresholdDegPerH": 0.05,
    "maxStrainMmPerM": 4.19,      "strainThresholdMmPerM": 3.00,
    "maxSubsidenceMm": 1513,
    "strainResolved": true,       // false ⇒ the array cannot resolve strain at all
    "packetDeliveryPct": 94.6, "uptimePct": 99.1, "healthy": false
  },
  "gatewayVolts": 13.2, "gatewayBatteryPct": 78, "storagePct": 85
}
```

11.1 Broadcast discipline

Ingest can accept hundreds of frames per second; rebuilding and serialising the full snapshot on each would spend most of the server's time producing JSON no human can perceive. Instead ingest marks the state dirty and a background task rebuilds at most **4 times a second**, coalescing bursts into one update. A keep-alive `{"type":"ping"}` goes out after 25 s of silence so idle proxies do not drop the socket.

12 Frontend architecture

12.1 Source selection

```
createSource(baseUrl)
   ├─ GET /api/health, 2.5 s timeout
   │     ok  →  LiveSource   — WebSocket + REST history
   │     no  →  SimulatedSource — the built-in physics model, and the header says so
```

This is not a development convenience. Offline capability is a requirement of the problem statement: a monitoring screen that goes blank when the link drops is worse than useless on a mine site. `LiveSource` reconnects with exponential backoff to a 15 s ceiling, and caches history responses for 4 s to avoid stampeding the backend on rapid range changes.

12.2 Panels

Component	What it shows
KPI row	Six tiles: node count, alerts, max tilt, **max ground strain** (or "—" with "Array too sparse to resolve" when `strainResolved` is false), packet delivery, system status
Leaflet map	Node pins coloured by risk band, mesh links with the active route highlighted, tooltips showing tilt and signed strain
Three.js scene	3-D subsidence terrain sampled from the physics surface
Deformation trend	Three stacked panels on one shared time axis: tilt (°), vibration (mg), **post temperature (°C)**
Params panel	Four gauges: pitch, roll, vibration RMS, ground strain
Alerts panel	Open alerts newest first, with ack and resolve
Prediction chart	Observed peak risk per day plus a three-day forecast

> **Why temperature is the third trend panel.** It is the one confound that can masquerade as ground movement: the post expands and the tilt trace rises with it. The plotted tilt is already corrected, so the panel exists to be *checked against* — an excursion that tracks the temperature curve is residual drift; one that ignores it is ground. Three different quantities on three different scales are drawn as small multiples rather than on shared or dual axes, because overlaying them would make arbitrary crossings look meaningful.

13 Simulator

The simulator is not a mock. It encodes genuine frames, routes them through the mesh model, and delivers them to the backend exactly as an ESP32 gateway would — over the same HTTP or MQTT transport, in the same base64 batches. The backend cannot tell the difference, which is the point: if the simulator can drive the stack, so can the field.

13.1 Physics — `physics.py`

The influence-function (Knothe) method, the standard analytical model for surface movement above underground extraction. A Gaussian influence function of radius `r = H / tan β`, integrated over the extracted panel, gives a complementary-error-function profile; the 2-D surface is the product of the two 1-D profiles. Analytic first and second derivatives are checked against finite differences, so an algebra slip cannot quietly poison the training data. The published results are reproduced: subsidence is exactly half of maximum above the panel edge, tilt peaks there and vanishes at the trough centre, strain is tensile at the rim and compressive over the goaf.

13.2 Sensor error model — `sensors.py`

Error source	Magnitude	Origin
LIS3DH tilt noise	`50 mdeg`	±2 g, 12-bit → ~1 mg/LSB, after on-node averaging
Thermal drift	`18 mdeg/°C`	Mounting-post expansion. The dominant error, and systematic
Bias random walk	`2 mdeg/sample`	Slow bias wander
Install offset	`σ ≈ 900 mdeg`	Hand-planted on uneven ground
Die temperature noise	`0.5 °C`	Only needs to track the change, not be absolute
GNSS horizontal	`2.5 m CEP`	NEO-6M. Three orders of magnitude from a subsidence signal
Ambient vibration	`12 ± 4 mg`	Wind, distant plant, background

State persists across reads, because that is what makes the data realistic: bias wanders, batteries deplete, and a node that develops a sensor fault keeps it. A memoryless noise model would be far easier for a model to learn than reality.

13.3 Scenarios

Name	What it trains against
`stable`	No extraction. The false-positive baseline
`slow_creep`	Gradual settlement over a long window
`accelerating`	Rate increasing — what the tilt-rate channel exists to catch
`sudden_collapse`	Rapid failure, including GNSS-visible gross displacement
`false_alarm`	**Blasting, plant traffic and thermal cycling.** All look like movement if you only threshold. This scenario exists specifically to train against crying wolf

13.4 Mesh model — `mesh.py`

LoRa link budget for the E220-900M22S: 22 dBm TX, −123 dBm sensitivity at SF9/125 kHz, 43 dB reference loss at 1 m, path-loss exponent **3.4** for ground-level nodes with obstructed Fresnel zones, 4.5 dB shadowing sigma. Delivery probability is a logistic curve over link margin — LoRa holds near-perfect delivery until it falls off a cliff within a few dB of its limit, which is exactly the behaviour that makes a mesh worth having.

> **The measured differentiator.** Under this propagation model with terrain obstructions in the way, a star topology delivers **43%** of frames and the mesh delivers **100%**. That is the claim as a number rather than a slogan — and it also says something practical for deployment: antenna height buys more range than transmit power does.

13.5 Virtual gateway — usage

```
python -m simulator.virtual_gateway [options]

  --api http://localhost:8000     backend base URL
  --site jharia                   site preset
  --scenario accelerating         stable|slow_creep|accelerating|sudden_collapse|false_alarm
  --layout transect               transect (21 nodes, resolves strain) | grid (~105 needed)
  --nodes 21                      transect node count
  --transport http                http | mqtt
  --interval 0.25                 real seconds per tick (1 tick = 15 simulated minutes)
  --ticks N                       stop after N ticks
  --batch 64                      max frames per POST
  --seed 7 --start-day 40.0
```

Frames carry **simulated** time, not the wall clock. A compressed run posts hours of scenario in seconds, and stamping that with the real clock would collapse every derived rate — tilt rate above all — into a division by nearly zero.

14 Deployment and configuration

14.1 Two shapes, one codebase

Shape	Database	Command
Local / demo / on-site	SQLite	`make api` — nothing to install
Deployed	TimescaleDB + PostGIS	`make up` — Docker Compose

The query layer is identical: `state.py` uses a correlated `max(time)` rather than a window function precisely so both dialects work. Only the hypertable DDL differs.

14.2 Make targets

```
make install     create the venv, install backend + ml + the shared proto package, npm install
make api         uvicorn on :8000            (terminal 1)  → /docs
make gateway     stream physics-backed frames (terminal 2)
make web         vite dev server on :5173    (terminal 3)
make demo        warm the DB with two simulated days, then stream live
make up/down     the deployed stack: TimescaleDB, Mosquitto, Redis, API, web
make logs        follow the deployed stack
make test        all three suites (217 tests)
make clean       remove the local database and build artefacts
```

14.3 Services in the deployed stack

Service	Image	Role
`db`	`timescale/timescaledb-ha:pg16`	Bundles PostGIS, so time-series and geospatial live in one database. Runs `migrations/` on first start
`mqtt`	`eclipse-mosquitto:2`	:1883 for gateways, :9001 websockets for browser/debug clients
`redis`	`redis:7-alpine`	Ephemeral, no persistence
`api`	`backend/Dockerfile`	Built from the repo root so it can install the shared proto package
`ml`	`ml/Dockerfile`	:8100, `ML_SERVICE_URL` already wired into the API
`web`	`web/Dockerfile`	:5173

14.4 Settings

```
DATABASE_URL        sqlite+aiosqlite:///./subnet.db     Compose overrides with Postgres
MQTT_HOST           (unset)                             unset ⇒ gateways use POST /api/ingest
MQTT_PORT           1883
MQTT_UPLINK_TOPIC   subnet/gw/+/up
MQTT_DOWNLINK_TOPIC subnet/gw/{gateway}/cmd
CORS_ORIGINS        http://localhost:5173,http://127.0.0.1:5173

── operator-facing thresholds, kept server-side so the dashboard and the app cannot disagree ──
TILT_THRESHOLD_DEG           0.60    10 mm/m, the NCB-style disruptive limit
STRAIN_THRESHOLD_MM_PER_M    3.00    NCB "appreciable damage" boundary
VIBRATION_THRESHOLD_MG     400.0
TILT_RATE_THRESHOLD_DEG_PER_H 0.05   the precursor trigger
TILT_DRIFT_MDEG_PER_C       18.0     post thermal expansion; measure per node on commissioning
TILT_RATE_WINDOW_HOURS       6.0
NODE_STALE_SECONDS           180     unheard-from longer than this ⇒ offline
```

SMS needs no credentials here: it is sent by the gateway's own SIM800L, not by a cloud provider. An SMS that requires the backend to be reachable is an SMS that fails exactly when you need it.

15 Test inventory

217 tests. The ones worth knowing about are those that assert a *limitation* — a system that only tests its happy path will overstate what the hardware can do.

15.1 `packages/subnet-proto` — 32 tests

Group	Asserts
Round trips	Every message type survives encode → decode unchanged, including southern/western hemispheres in signed GNSS coordinates
Corruption	Bad magic, bad version, truncated payload, flipped CRC bit — all raise `ProtocolError`, never a crash
Airtime	A telemetry frame is exactly 34 B, so the SF9 budget holds
Packing	Fix quality and satellite count share one byte without bleeding; a 70-satellite count saturates at 63 instead of overflowing into the fix bits
C ↔ Python	Compiles `mesh_proto.h`, compares every struct size and a fully built frame byte for byte. **This is what stops firmware and backend drifting apart**

15.2 `ml` — 122 tests

Group	Asserts
Physics	Analytic derivatives match finite differences; subsidence is half of maximum at the panel edge; tilt peaks at the rim and vanishes at the trough centre; strain is tensile at the rim, compressive over the goaf
Sensors	Install offset is applied; extreme tilt clamps rather than wrapping an int16 into a huge opposite-sign reading
Thermal	A 15 °C swing with no ground movement shifts apparent tilt by >5× the sensor noise — and the reported temperature corrects it back to within 1 mdeg. Residual with realistic temperature noise stays under the tilt noise
GNSS limits	A 10 mm displacement is *lost inside* the receiver's own noise; a 30 m displacement is recovered to ±3 m. The limitation is asserted, not hand-waved
Mesh	Star delivers 43%, mesh delivers 100%; self-healing when a relay dies
Wire safety	Every reading the sensor model can emit fits the wire format, across temperature and tilt extremes

15.3 `backend` — 63 tests

Group	Asserts
Ingest	A corrupt frame is rejected without being fatal, and a good frame alongside it still lands; a replayed frame does not duplicate
Baselines	The first frame never alerts; movement after it is measured; history and gauges agree; recalibration re-zeros
Thermal drift	A pure temperature swing raises no alert; real movement *during* a swing still reads at full magnitude — the correction must not swallow the signal with the drift
Reconstruction	Strain correlation >0.95 against the analytic field at the required spacing, and **<0.5 at radio spacing** — the negative result is pinned so the node count cannot be quietly optimised down. Subsidence within 10% when anchored, flagged when not
Transect	21 nodes recover strain (>0.95) and the subsidence profile (<5% error), both ends anchored, crossing both rims. Too few nodes warns rather than silently under-resolving
Layout mirror	The spacing rule stated in the field planner and in the reconstruction agree — if they drift, a field gets built to a spacing the reconstruction cannot use
Degraded input	A lone node reports `strainValid: false`, not zero strain; coincident nodes do not divide by zero; an empty field is not an error
GNSS	A good fix self-places an unsurveyed node; a 110 m move alerts; metre-scale wander does not; a 2-D-only fix is ignored
Alerts	Threshold breach raises once, rate-limited per node; node events raise immediately; ack and resolve
Downlink	Config push creates a pending version whose hash matches what the node will verify; the ACK closes the loop

16 Build status and open work

Component	Status	Where
Wire protocol, Python + C, byte-identical	BUILT	`packages/subnet-proto, firmware/common`
Physics, sensor error model, labelled scenarios	BUILT	`ml/simulator`
Mesh routing model and delivery statistics	BUILT	`ml/simulator/mesh.py`
Virtual gateway	BUILT	`ml/simulator/virtual_gateway.py`
Field reconstruction — strain and subsidence from tilt	BUILT	`backend/app/deformation.py`
Ingest, risk scoring, alerting, downlink, snapshot, WebSocket	BUILT	`backend/app`
Web dashboard — Leaflet 2-D, Three.js 3-D, live feed	BUILT	`web/src`
Deployed stack — TimescaleDB/PostGIS, Mosquitto, Redis	BUILT	`docker-compose.yml`
Node firmware	TO DO	`firmware/node/src`
Gateway firmware, local rule engine, SIM800L driver	TO DO	`firmware/gateway/src`
ML training pipeline	TO DO	`ml/training`
ML inference service	TO DO	`ml/service`
SMS dispatcher	TO DO	`column exists, no sender`
Android app	TO DO	`android/`

16.1 Suggested order

1. **ML inference service** — the “AI-enabled” claim. Port and `ML_SERVICE_URL` are already wired; the labelled data generator is ready. The strongest form is not a classifier over summary statistics but an *estimator of the deformation field*: fit the Knothe profile — already available analytically, with validated derivatives — to all 21 noisy tilt readings and differentiate the fit, so every node constrains the curvature rather than only its two neighbours.
2. **SMS dispatcher** — the alerting claim. `alerts.notified_sms` exists as the idempotency flag; both the gateway-local and server-directed paths need writing.
3. **Android app** — consumes the same `/api/snapshot` document, so there is no new contract to design.
4. **Firmware** — the protocol, frame layout and config schema are all fixed, so the contract is settled. Note the E220's SPI interface means driving the LLCC68 at register level (RadioLib), not a UART transparent mode.

16.2 Known limitations, stated plainly

- **Nothing here has run on real hardware.** The protocol is proven against a compiled C header and the pipeline against a physics-backed simulator, but no ESP32 has been flashed.
- **The 18 mdeg/°C drift coefficient is a design default.** It should be measured per node during a commissioning thermal soak. There is a settings knob (`TILT_DRIFT_MDEG_PER_C`) but not yet per-node calibration storage.
- **The forecast is a trend extrapolation**, and says so in its own docstring. The value to an operator is the lead time it implies, and that value is only real if the operator can tell when to distrust it — an honest simple model beats an opaque one that cannot be sanity-checked on a night shift.
- **Strain is an hourly product, not a per-frame one.** At 150 m spacing single-sample strain noise is ~0.49 mm/m — the entire width of the NCB “negligible” band. Averaging over an hour brings it to 0.064 mm/m. This is physically appropriate anyway, since strain damage accumulates over days.
- **GNSS cannot measure subsidence**, and no amount of processing will change that. It localises, disciplines the clock, supplies the baselines strain divides by, and catches gross displacement.